# Reproducible Knowledge Distillation Pipelines for Recommendation with RecDistillery*

Leonardo Di Gioia[1,*], Marialuisa Pisicchio[1,*], Alberto Carlo Maria Mancino[1,*], Daniele Malitesta[2], Fedelucio Narducci[1] and Tommaso Di Noia[1]

[1]Politecnico di Bari, Bari, Italy
[2]LUISS Guido Carli University, Rome, Italy

**Abstract**

Reusing knowledge distillation methods in recommender systems remains difficult because existing implementations are often tied to specific model architectures, data pipelines, and evaluation protocols. Moving a method to a different setting therefore requires substantial adaptation and complicates fair comparison. We present RecDistillery, an open-source framework that supports reproducible teacher–student experiments within a shared workflow. RecDistillery separates the knowledge supplied by a teacher from its original implementation through a common interface exposing rankings, prediction scores, or latent representations. This design accommodates both integrated PyTorch-based models and externally trained recommenders, while reusable distillation components avoid rebuilding the experimental pipeline for each method. Standardized data preparation, configuration-driven optimization, and consistent evaluation connect these components into a common experimental environment. Four case studies on CiteULike-a involve teachers from different reproducibility frameworks and an externally trained ItemKNN model. They demonstrate how RecDistillery supports heterogeneous teacher–student combinations and show that distillation gains depend on the selected models and transfer strategy. By recording configurations, model artifacts, evaluation outputs, and timing information, RecDistillery supports the inspection, reproduction, and extension of recommendation experiments. Code, documentation, and executable examples are publicly available to facilitate reuse.

**Keywords**

Recommender systems, Knowledge distillation, Reproducibility, Interoperability, RecDistillery

## 1. RecDistillery and the reproducibility problem

Knowledge distillation transfers knowledge from a teacher model to a student and has been studied for training compact recommender systems [2, 3]. Yet, studying this transfer involves much more than simply choosing a distillation objective. Teacher and student models must be integrated within a common experimental workflow, data must be processed consistently, and all models must be evaluated under the same protocol. When these components are scattered across separate research codebases, reproducing an existing method or conducting a fair comparison with alternative approaches can require considerable implementation effort. This challenge reflects a broader and well-documented reproducibility issue in recommender systems research [4, 5].

RecDistillery organizes these steps into a reusable experimental environment. Its contribution is a common infrastructure for connecting recommendation backbones and distillation strategies while retaining the implementations provided by existing ecosystems. The framework brings together data preparation, teacher integration, student optimization, and evaluation, and records the configuration and artifacts of each run. It thereby supports both reproducing existing methods and prototyping new teacher–student combinations. In this paper, we present RecDistillery as a modular framework

*IIR 2026: 16th Italian Information Retrieval Workshop, November 12–14, 2026, Rome, Italy*

*This paper is an extended abstract of the article "RecDistillery: A Framework for Teacher-Student Knowledge Distillation in Recommender Systems" (Proceedings of ACM CIKM 2026) [1].

*Corresponding authors.
l.digioia4@phd.poliba.it (L. D. Gioia); m.pisicchio2@studenti.poliba.it (M. Pisicchio); alberto.mancino@poliba.it (A. C. M. Mancino); dmalitesta@luiss.it (D. Malitesta); fedelucio.narducci@poliba.it (F. Narducci); tommaso.dinoia@poliba.it (T. D. Noia)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

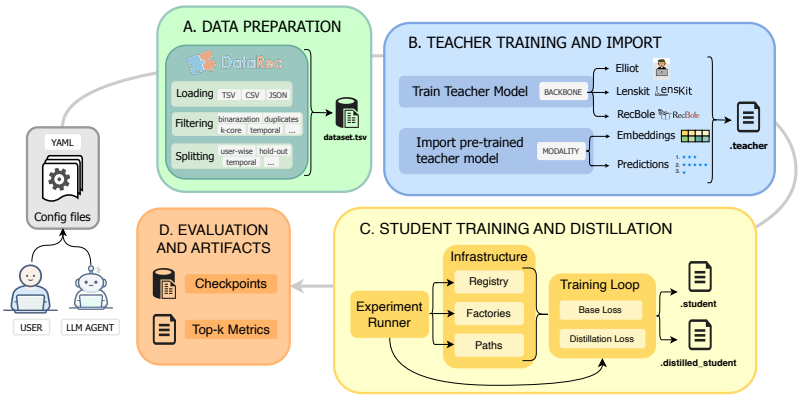



**Figure 1:** The RecDistillery architecture. Data preparation, teacher integration, student distillation, and evaluation are connected through shared interfaces and YAML configurations. The `.teacher` representation exposes the knowledge used by the distillation pipeline.

for reproducible knowledge distillation in recommender systems, focusing on interoperability across recommendation ecosystems and on the workflows enabled by a shared teacher–student pipeline.

## 2. Interoperability as the organizing principle

In teacher–student knowledge distillation, a compact student model learns from the predictions or representations of a more expressive teacher, together with the original recommendation data. RecDistillery separates the information supplied by a teacher from the software used to implement it. Each teacher is exposed through a common `.teacher` abstraction that can provide recommendation lists, full or top-$k$ prediction scores, latent user–item representations, or combinations of these signals. A distillation method consumes the signals available through this interface, without requiring the student training code to depend on the teacher's original implementation. Figure 1 places this interface within the overall workflow.

Teachers can be incorporated into RecDistillery in different ways depending on how they were originally implemented and trained. Models supported through dedicated adapters remain fully integrated with the framework, which can handle their training and tuning before exporting them for use in distillation experiments. This integration covers 17 PyTorch-based implementations drawn from RecBole [6], Elliot [7], and LensKit [8]. Together, these models cover nine backbone families and can be used on either side of the teacher–student setting.

The framework is not limited, however, to models trained within these ecosystems. Teachers developed externally can also be incorporated through compatible checkpoints or precomputed outputs, making it possible to reuse models whose original training procedure lies outside RecDistillery, including non-PyTorch implementations. What matters in this case is that the information required by the selected distillation method can be recovered in a compatible form.

This separation is important for reuse because it allows RecDistillery to preserve native model implementations when direct integration is possible without excluding teachers produced elsewhere. Compatibility checks and a shared internal representation bridge these different sources, so that externally trained and natively supported teachers can ultimately feed into the same distillation workflow.

## 3. Controlling the distillation experiment

Within RecDistillery, a student backbone is paired with a distiller that converts teacher information into an auxiliary training signal. The student jointly optimizes a recommendation objective and a distillation objective,

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{KD}}, \tag{1}$$

where $\lambda$ controls the contribution of transferred knowledge. The same training loop supports different forms of supervision through the teacher interface. RecDistillery includes methods from DE-RRD [9], which transfer representation and ranking information; Topology Distillation [10], which transfers relations in the embedding space; and UnKD [11], which distills within popularity-aware item groups.

Control of the experiment also depends on the surrounding data and evaluation procedures. RecDistillery uses DataRec [12] for preprocessing, splitting, and loading interactions into standardized training, validation, and test partitions. Example preparation scripts cover CiteULike-a, Amazon CDs & Vinyl 2014, and BookCrossing; other datasets can be incorporated through DataRec. Teacher and student models are evaluated with the same ranking metrics, including Precision, Recall, nDCG, and Hit Rate. Validation measurements also guide checkpoint selection and early stopping.

YAML configuration files specify datasets, models, optimization settings, evaluation protocols, and distillation parameters. These definitions allow experimental components to be recombined without modifying the training pipeline. Grid search and Bayesian optimization are supported, while a Markdown specification describes the configuration schema for human- and agent-assisted preparation of experiments. Each RecDistillery run receives a unique identifier and retains its configuration, checkpoints, logs, and evaluation outputs. Recorded timing information includes training duration, average epoch time, and distillation overhead, making experimental artifacts available for inspection and comparison. These training-time records do not provide measurements of inference latency or memory usage.

## 4. Evidence from heterogeneous teacher–student pairs

We evaluate four representative RecDistillery configurations on CiteULike-a [13]. After preprocessing with DataRec and iterative 5-core filtering on users and items, the dataset contains 5,536 users, 15,429 items, and 200,179 interactions. In the three embedding-based comparisons, teachers use 200 latent factors and students use 20, corresponding to a 90% reduction in embedding dimensionality. This percentage does not quantify a reduction in total memory usage or inference latency. Distillation hyperparameters are selected through 30 Bayesian-optimization trials. Training runs for up to 300 epochs, with early stopping based on validation nDCG@20.

Table 1 reports recommendation performance for the four case studies. The first three configurations connect integrated teacher and student models from Elliot, LensKit, and RecBole. The fourth imports an external ItemKNN teacher and transfers its knowledge to a NeuMF student from Elliot. It demonstrates cross-architecture knowledge transfer between a non-PyTorch teacher and a neural student within the RecDistillery workflow.

**Table 1**
Performance of teacher, non-distilled student, and distilled student models on CiteULike-a. All metrics are computed at rank 20. Student denotes training without distillation; DE and RRD are the components of DE-RRD, and FTD denotes Full Topology Distillation.

| Ecosystem | Backbone | Training | Precision | Recall | nDCG | Hit Rate |
|---|---|---|---|---|---|---|
| Elliot | LGCN | Teacher | 0.0862 | 0.2456 | 0.1933 | 0.6882 |
| | | Student | 0.0628 | 0.1775 | 0.1353 | 0.5952 |
| | | DE | 0.0649 | 0.1849 | 0.1423 | 0.6026 |
| LensKit | BPRMF | Teacher | 0.0826 | 0.2320 | 0.1842 | 0.6544 |
| | | Student | 0.0544 | 0.1500 | 0.1130 | 0.5332 |
| | | FTD | 0.0540 | 0.1492 | 0.1139 | 0.5385 |
| RecBole | SGL | Teacher | 0.0876 | 0.2547 | 0.2015 | 0.6964 |
| | | Student | 0.0578 | 0.1686 | 0.1293 | 0.5744 |
| | | RRD | 0.0607 | 0.1693 | 0.1324 | 0.5704 |
| External | ItemKNN | Teacher | 0.0786 | 0.2173 | 0.1753 | 0.6172 |
| Elliot | NeuMF | Student | 0.0438 | 0.1215 | 0.0854 | 0.4612 |
| | | UnKD | 0.0332 | 0.0895 | 0.0721 | 0.3819 |
| | | RRD | 0.0503 | 0.1347 | 0.1011 | 0.4879 |

The reported metric differences vary across teacher–student pairs and transfer strategies. DE improves all four metrics over the non-distilled LGCN student in Elliot. For LensKit/BPRMF, FTD improves nDCG and Hit Rate, but slightly decreases Precision and Recall. RRD improves Precision, Recall, and nDCG for RecBole/SGL, while Hit Rate decreases. These differences motivate evaluating distilled models with a common set of metrics rather than assuming that an improvement in one measure implies an improvement in all others.

The external ItemKNN teacher provides a complementary example. With the same NeuMF student, RRD raises nDCG@20 from 0.0854 to 0.1011 and improves the other three metrics, whereas UnKD reduces nDCG@20 to 0.0721 and also decreases Precision, Recall, and Hit Rate. This case demonstrates that knowledge from a non-PyTorch recommender can be used within the RecDistillery training pipeline, while its effectiveness still depends on the selected distillation method.

These case studies demonstrate the workflows supported by RecDistillery on CiteULike-a; they do not establish a general ranking of distillation methods or generalization across datasets. The reported metric differences are descriptive: multi-seed variability estimates and statistical significance tests are not provided. Inference latency and memory usage are also not reported, so these results do not establish deployment-efficiency gains.

## 5. Conclusions and Future Work

We presented RecDistillery as a common infrastructure for teacher integration, student training, knowledge distillation, and evaluation in recommender systems. By separating teacher knowledge from its original implementation and recording experimental configurations and artifacts, the framework supports reproducible workflows with both integrated PyTorch models and externally trained recommenders. A broader evaluation should use explicitly documented tuning budgets, additional datasets [14], repeated runs with different random seeds, and statistical tests of metric differences. Profiling inference latency and memory usage is also needed to assess deployment efficiency. Planned extensions include sequential [15] and multimodal recommendation, differential privacy [16], graph-based recommenders [17] and online distillation. Source code, documentation, and executable examples are accessible through https://github.com/sisinflab/RecDistillery. The repository includes the `recdistillery.ipynb` notebook for reproducing the case studies through Google Colab. These resources provide a starting point for extending RecDistillery and sharing reproducible knowledge distillation experiments with the recommendation community.

## Declaration on Generative AI

During the preparation of this work, the authors used OpenAI Codex for paraphrasing and rewording, grammar and spelling checks, and assistance with manuscript organization and LaTeX formatting. The authors retain full responsibility for the scientific content and the final manuscript.

## Acknowledgments

The authors acknowledge partial support from the following projects: “LIFE Project: the itaLian system wide Frailty nEtwork” (local code: T2-AN-12; CUP: D93C22000640001); “Research Actions for Reducing the Impact on Agricultural and Natural Ecosystems of the Harmful Plant Pathogen Xylella fastidiosa” (REACH-XY; CUP: B93C22001920001); “Advanced Cooperative SOC” (ACSOC), funded by the European Union’s Digital Europe Programme (Grant Agreement No. 101190248); Grifo Multimedia – “LUtech Campus Ecosystem – LUCE” (Application Code: 22ROJB5; CUP: B95I25000160007); and “AURA – Augmented Unified Resource for Agents” (Applicant: KONECTA ITALIA S.P.A.; Application Code: CDP001572). The authors also acknowledge ISCRA for awarding access to the LEONARDO supercomputer, hosted by CINECA (Italy).

## References


[1] M. Pisicchio, A. C. M. Mancino, L. Di Gioia, D. Malitesta, F. Narducci, T. Di Noia, RecDistillery: A framework for teacher-student knowledge distillation in recommender systems, in: Proceedings of the 35th ACM International Conference on Information and Knowledge Management, CIKM 2026, Rome, Italy, November 07-11, 2026, ACM, 2026. doi:10.1145/3799682.3840179.

[2] G. E. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, CoRR abs/1503.02531 (2015).

[3] S. Li, H. Guo, X. Tang, R. Tang, L. Hou, R. Li, R. Zhang, Embedding compression in recommender systems: A survey, ACM Comput. Surv. 56 (2024) 130:1–130:21. doi:10.1145/3637841.

[4] M. Ferrari Dacrema, S. Boglio, P. Cremonesi, D. Jannach, A troubling analysis of reproducibility and progress in recommender systems research, ACM Trans. Inf. Syst. 39 (2021) 20:1–20:49.

[5] D. Malitesta, A. C. M. Mancino, C. Pomo, V. W. Anelli, A. Bellogín, T. D. Noia, On the impact of graph neural networks in recommender systems: A topological perspective, Found. Trends Inf. Retr. (2027). In press.

[6] W. X. Zhao, S. Mu, Y. Hou, Z. Lin, Y. Chen, X. Pan, K. Li, Y. Lu, H. Wang, C. Tian, Y. Min, Z. Feng, X. Fan, X. Chen, P. Wang, W. Ji, Y. Li, X. Wang, J.-R. Wen, RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, Association for Computing Machinery, New York, NY, USA, 2021, pp. 4653–4664.

[7] V. W. Anelli, A. Bellogín, A. Ferrara, D. Malitesta, F. A. Merra, C. Pomo, F. M. Donini, T. Di Noia, Elliot: A comprehensive and rigorous framework for reproducible recommender systems evaluation, in: SIGIR, ACM, 2021, pp. 2405–2414. doi:10.1145/3404835.3463245.

[8] M. D. Ekstrand, Lenskit for python: Next-generation software for recommender systems experiments, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, Association for Computing Machinery, New York, NY, USA, 2020, pp. 2999–3006.

[9] S. Kang, J. Hwang, W. Kweon, H. Yu, DE-RRD: A knowledge distillation framework for recommender system, in: CIKM, ACM, 2020, pp. 605–614.

[10] S. Kang, J. Hwang, W. Kweon, H. Yu, Topology distillation for recommender system, in: KDD, ACM, 2021, pp. 829–839.

[11] G. Chen, J. Chen, F. Feng, S. Zhou, X. He, Unbiased knowledge distillation for recommendation, in: WSDM, ACM, 2023, pp. 976–984.

[12] A. C. M. Mancino, S. Bufi, A. D. Fazio, A. Ferrara, D. Malitesta, C. Pomo, T. D. Noia, Datarec: A python library for standardized and reproducible data management in recommender systems, in: SIGIR, ACM, 2025, pp. 3478–3487.

[13] C. Wang, D. M. Blei, Collaborative topic modeling for recommending scientific articles, in: KDD, ACM, 2011, pp. 448–456.

[14] V. Paparella, A. C. M. Mancino, A. Ferrara, C. Pomo, V. W. Anelli, T. D. Noia, Knowledge graph datasets for recommendation, in: KaRS@RecSys, volume 3560 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 109–117.

[15] C. Mallamaci, A. V. Petrov, A. C. M. Mancino, V. W. Anelli, T. D. Noia, C. Macdonald, Balancing accuracy and novelty with sub-item popularity, in: RecSys, ACM, 2025, pp. 1137–1141.

[16] A. Ferrara, A. D. Fazio, A. C. M. Mancino, T. D. Noia, E. D. Sciascio, Enhancing utility in differentially private recommendation data release via exponential mechanism, in: ECIR (3), volume 15574 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 34–51.

[17] S. Bufi, A. C. M. Mancino, A. Ferrara, D. Malitesta, T. D. Noia, E. D. Sciascio, KGUF: simple knowledge-aware graph-based recommender with user-based semantic features filtering, in: IRonGraphs, volume 2197 of *Communications in Computer and Information Science*, Springer, 2024, pp. 41–59.